🟢 LEVEL 1: Must Complete (Core Remaining Work) These are the real pending engineering parts. 1. Cache Invalidation Redis is added

✅ Now maintain data correctness. Without this → wrong product price/stock.

You already **added Redis + caching**.
Now we make it **production safe** by handling **cache invalidation**.

Let's do this step by step.

✅ First Understand the Problem (Very Important)

Right now your flow is like this:

```
Client → Product Service
          ↓
       Redis Cache
          ↓ (miss)
        Database
```

When product is fetched:

```
GET /products/{id}
→ stored in Redis
```

Good.

But problem starts here:

```
UPDATE product price
UPDATE stock
DELETE product
```

Database changes ✅
Redis still has OLD DATA ❌

Result:

- Wrong price shown
- Wrong stock
- Order failures
- Real production bug

So whenever data changes → **cache must be cleared or updated**.

✅ Cache Invalidation Strategy (Used in Real Systems)

We will use **Cache Aside Pattern**.

Meaning:

Operation	Cache Action
GET Product	Read from cache
CREATE Product	Clear product cache
UPDATE Product	Clear product cache
DELETE Product	Clear product cache

Simple. Reliable. Industry standard.

✅ Step 1: Check Your Cache Name

You probably have something like:

```
@Cacheable(value = "products", key = "#id")
public Product getProductById(Long id)
```

👉 Cache name = "products"

Remember this.

✅ Step 2: Invalidate Cache After Update

ProductServiceImpl.java

UPDATE Product

Add:

```
@Override
@CacheEvict(value = "products", key = "#id")
public ProductResponseDto updateProduct(Long id,
                                        ProductRequestDto productRequestDto) {

    log.info("Updating product with id: {}", id);

    Product existing = productRepository.findById(id)
            .orElseThrow(() -> {
                log.warn("Cannot update. Product not found with id: {}", id);
                return new ResourceNotFoundException(
                        "Product not found with id: " + id);
            });

    existing.setName(productRequestDto.getName());
    existing.setPrice(productRequestDto.getPrice());
    existing.setStock(productRequestDto.getStock());
```

```
    Category category = categoryRepository
            .findById(productRequestDto.getCategoryId())
            .orElseThrow(() ->
                    new ResourceNotFoundException(
                            "Category not found with id: "
                                    + productRequestDto.getCategoryId()));

    SubCategory subCategory = subCategoryRepository
            .findById(productRequestDto.getSubCategoryId())
            .orElseThrow(() ->
                    new ResourceNotFoundException(
                            "SubCategory not found with id: "
                                    + productRequestDto.getSubCategoryId()));

    if (!subCategory.getCategory().getId()
            .equals(category.getId())) {
        throw new ResourceNotFoundException(
                "SubCategory does not belong to Category");
    }

    existing.setCategory(category);
    existing.setSubCategory(subCategory);

    Product updated = productRepository.save(existing);

    ProductResponseDto responseDto =
            modelMapper.map(updated, ProductResponseDto.class);

    responseDto.setCategoryId(category.getId());
    responseDto.setCategoryName(category.getName());

    responseDto.setSubCategoryId(subCategory.getId());
    responseDto.setSubCategoryName(subCategory.getName());

    responseDto.setImageUrl(
            buildImageUrl(updated.getImagePath()));

    return responseDto;
}
```

✅ What Your Code Is Doing Correctly

You added:

```
@CacheEvict(value = "products", key = "#id")
```

on:

updateProduct(Long id, ProductRequestDto productRequestDto)

✓ **Flow now becomes**

1. Product exists in Redis cache
2. UPDATE API called
3. Database updated
4. Redis cache entry removed automatically
5. Next GET → fresh data loaded

Exactly what we want.

✅ Why This Works Properly

Spring executes cache eviction **after successful method execution**.

So:

✅ If update succeeds → cache cleared
✅ If exception occurs → cache NOT removed

This prevents accidental cache loss.

Your validations and exceptions are already solid, so behavior is safe.

✅ One Important Hidden Check (Most Developers Miss)

Your @Cacheable method should look like this:

```
@Cacheable(value = "products", key = "#id")
public ProductResponseDto getProductById(Long id)
```

👉 Cache name must match:

products ✅
products ✅

If names differ even slightly:

product
products
productCache

❌ eviction will NOT work.

So confirm this once.

✅ Redis Cache Invalidation Notes

(Spring Boot CacheEvict & beforeInvocation)

---

1. Why Cache Invalidation is Required

In an application using Redis caching, data is stored temporarily to improve performance.

Example:

GET /products/1

First request:

Database → Redis Cache

Next requests:

Redis → Response (No DB hit)

This improves response time and reduces database load.

Problem Without Cache Invalidation

When product data changes:

UPDATE product price
UPDATE stock
DELETE product

Database gets updated but Redis may still contain old data.

Result:

- Wrong product price shown
- Incorrect stock information
- Data inconsistency between APIs

Therefore, cache must be cleared whenever data changes.

---

2. Cache Eviction Using @CacheEvict

Spring Boot provides @CacheEvict to remove cached data automatically.

Example

```
@CacheEvict(value = "products", key = "#id")
public ProductResponseDto updateProduct(Long id,
                                        ProductRequestDto dto) {
    // update logic
}
```

How It Works

If product cache exists:

products::1 → iPhone | ₹80000

After successful update:

products::1 → removed from Redis

Next request:

GET /products/1
→ Database hit
→ Fresh value stored again in Redis

This ensures data correctness.

---

3. Execution Timing of Cache Eviction

Spring must decide **when cache should be removed**.

This is controlled using:

beforeInvocation

---

4. Default Behaviour (Recommended)

Default configuration:

```
@CacheEvict(value = "products",
            key = "#id",
            beforeInvocation = false)
```

Even if not written explicitly, Spring uses this behavior.

Execution Flow

Successful Update

Step 1 → Method execution starts
Step 2 → Database updated successfully
Step 3 → Method completes
Step 4 → Cache removed

✅ Cache cleared only after successful DB update.

Update Failure

If exception occurs:

Step 1 → Method execution starts
Step 2 → Exception occurs
Step 3 → Method fails
Step 4 → Cache NOT removed

✅ Existing cache remains safe.

This prevents unnecessary cache loss.

---

5. beforeInvocation = true (Not Recommended Here)

Example:

```
@CacheEvict(value = "products",
            key = "#id",
            beforeInvocation = true)
```

Execution Flow

Step 1 → Cache removed immediately
Step 2 → Method executes
Step 3 → Database update fails

Result:

- Database still contains old data
- Cache already deleted
- Extra database calls generated later

This causes performance instability.

---

6. Recommended Rule

For **UPDATE and DELETE operations**:

✅ Use default behavior

beforeInvocation = false

Meaning:

Cache should be cleared only after successful database modification.

---

7. Cache Consistency with Multiple Caches

Applications often cache:

1. Single product
2. Product list

Example:

```
@Cacheable(value = "products", key = "#id")
getProductById()
```

```
@Cacheable("productList")
getAllProducts()
```

Problem

Updating one product affects both caches.

If only single product cache is removed:

Product Details → Updated ✅
Product List → Old Data ❌

---

8. Correct Solution Using @Caching

```
@Caching(evict = {
    @CacheEvict(value = "products", key = "#id"),
    @CacheEvict(value = "productList", allEntries = true)
})
```

Meaning

Annotation	Action
products cache	Removes updated product
productList cache	Clears entire product list

---

9. Final Industry Practice

Whenever caching includes:

Single Entity + Collection/List

Any modification must invalidate both caches to maintain consistency.

This pattern is widely used in ecommerce and distributed systems.

✅ Summary

- Redis improves performance using cached data
- Cache must be invalidated after data modification
- @CacheEvict removes outdated cache
- Default eviction happens after successful execution
- Avoid beforeInvocation = true for update/delete APIs

- Invalidate both object and list caches when needed

✅ Step 3: Invalidate Cache After Delete

```
@CacheEvict(value = "products", key = "#id")
@Override
public void deleteProduct(Long id) {

    log.info("Deleting product with id: {}", id);

    Product existing = productRepository.findById(id)
            .orElseThrow(() -> {
                log.warn("Cannot delete. Product not found with id: {}", id);
                return new ResourceNotFoundException("Product not found with id: " + id);
            });

    // ✅ delete primary image from S3
    deleteOldImageIfExists(existing);

    // ✅ delete gallery images from S3
    List<ProductImage> images = productImageRepository.findByProductId(existing.getId());

    for (ProductImage img : images) {
        if (img.getImagePath() != null) {
            s3Service.deleteFileByUrl(img.getImagePath());
        }
        if (img.getThumbnailPath() != null) {
            s3Service.deleteFileByUrl(img.getThumbnailPath());
        }
    }

    productRepository.delete(existing);

    log.info("Product deleted successfully with id: {}", id);
}
```

✅ **Step 4: Handle Product Creation**

When new product added, product list cache becomes invalid.

If you cache list APIs:

```
@Override
@CacheEvict(value = "productList", allEntries = true)
public ProductResponseDto addProduct(ProductRequestDto productRequestDto) {

    log.info("Creating new product with name: {}", productRequestDto.getName());

    Product product = modelMapper.map(productRequestDto, Product.class);
    product.setId(null);

    Category category = categoryRepository.findById(productRequestDto.getCategoryId())
            .orElseThrow(() -> new ResourceNotFoundException(
                    "Category not found with id: " + productRequestDto.getCategoryId()
            ));

    SubCategory subCategory = subCategoryRepository.findById(productRequestDto.getSubCategoryId())
            .orElseThrow(() -> new ResourceNotFoundException(
                    "SubCategory not found with id: " + productRequestDto.getSubCategoryId()
            ));

    if (!subCategory.getCategory().getId().equals(category.getId())) {
        throw new ResourceNotFoundException(
                "SubCategory id " + productRequestDto.getSubCategoryId() +
                        " does not belong to Category id " + productRequestDto.getCategoryId()
        );
    }

    product.setCategory(category);
    product.setSubCategory(subCategory);

    Product savedProduct = productRepository.save(product);

    log.info("Product created successfully with id: {}", savedProduct.getId());

    ProductResponseDto responseDto = modelMapper.map(savedProduct, ProductResponseDto.class);

    responseDto.setCategoryId(category.getId());
    responseDto.setCategoryName(category.getName());

    responseDto.setSubCategoryId(subCategory.getId());
    responseDto.setSubCategoryName(subCategory.getName());

    responseDto.setImageUrl(buildImageUrl(savedProduct.getImagePath()));

    return responseDto;
}

@Override
```

```java
@Cacheable("productList")
public List<ProductResponseDto> getAllProducts() {

    log.info("Fetching all products");

    List<Product> products = productRepository.findAll();

    log.info("Total products found: {}", products.size());

    return products.stream()
            .map(product -> {

                ProductResponseDto dto = modelMapper.map(product, ProductResponseDto.class);

                dto.setImageUrl(buildImageUrl(product.getImagePath()));

                if (product.getCategory() != null) {
                    dto.setCategoryId(product.getCategory().getId());
                    dto.setCategoryName(product.getCategory().getName());
                }

                if (product.getSubCategory() != null) {
                    dto.setSubCategoryId(product.getSubCategory().getId());
                    dto.setSubCategoryName(product.getSubCategory().getName());
                }

                return dto;
            })
            .collect(Collectors.toList());
}
```

Why allEntries=true?

Because list cache contains multiple products.

✅ **Final Flow After Fix**

First Call

GET product
→ DB hit
→ Stored in Redis

Update Product

PUT product
→ DB updated
→ Redis entry removed

Next GET

Cache miss
→ DB fresh data
→ Redis updated again

Perfect consistency ✅

FINAL CODE SOME PLACES WE HAVE ADDED Cache

```
package com.advann.product_service.service.serviceImpl;

import com.advann.product_service.dto.PagedResponseDto;
import com.advann.product_service.dto.ProductImageResponseDto;
import com.advann.product_service.dto.ProductRequestDto;
import com.advann.product_service.dto.ProductResponseDto;
import com.advann.product_service.entity.Category;
import com.advann.product_service.entity.Product;
import com.advann.product_service.entity.ProductImage;
import com.advann.product_service.entity.SubCategory;
import com.advann.product_service.exceptions.InvalidFileException;
import com.advann.product_service.exceptions.ResourceNotFoundException;
import com.advann.product_service.repository.CategoryRepository;
import com.advann.product_service.repository.ProductImageRepository;
import com.advann.product_service.repository.ProductRepository;
import com.advann.product_service.repository.SubCategoryRepository;
import com.advann.product_service.service.services.ProductService;
import com.advann.product_service.service.services.S3Service;
import jakarta.transaction.Transactional;
import lombok.RequiredArgsConstructor;
import net.coobird.thumbnailator.Thumbnails;
import org.modelmapper.ModelMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.Caching;
import org.springframework.data.domain.*;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
```

```
import java.io.ByteArrayOutputStream;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {

    private static final Logger log = LoggerFactory.getLogger(ProductServiceImpl.class);

    private final ProductRepository productRepository;
    private final ModelMapper modelMapper;
    private final CategoryRepository categoryRepository;
    private final SubCategoryRepository subCategoryRepository;
    private final ProductImageRepository productImageRepository;
    private final S3Service s3Service;

    @Value("${app.base-url}")
    private String baseUrl;

    private static final int MAX_GALLERY_IMAGES = 5;

    @Override
    @CacheEvict(value = "productList", allEntries = true)
    public ProductResponseDto addProduct(ProductRequestDto productRequestDto) {

        log.info("Creating new product with name: {}", productRequestDto.getName());

        Product product = modelMapper.map(productRequestDto, Product.class);
        product.setId(null);

        Category category = categoryRepository.findById(productRequestDto.getCategoryId())
                .orElseThrow(() -> new ResourceNotFoundException(
                        "Category not found with id: " + productRequestDto.getCategoryId()
                ));

        SubCategory subCategory = subCategoryRepository.findById(productRequestDto.getSubCategoryId())
                .orElseThrow(() -> new ResourceNotFoundException(
                        "SubCategory not found with id: " + productRequestDto.getSubCategoryId()
                ));

        if (!subCategory.getCategory().getId().equals(category.getId())) {
            throw new ResourceNotFoundException(
                    "SubCategory id " + productRequestDto.getSubCategoryId() +
                            " does not belong to Category id " + productRequestDto.getCategoryId()
            );
```

```
    }

    product.setCategory(category);
    product.setSubCategory(subCategory);

    Product savedProduct = productRepository.save(product);

    log.info("Product created successfully with id: {}", savedProduct.getId());

    ProductResponseDto responseDto = modelMapper.map(savedProduct, ProductResponseDto.class);

    responseDto.setCategoryId(category.getId());
    responseDto.setCategoryName(category.getName());

    responseDto.setSubCategoryId(subCategory.getId());
    responseDto.setSubCategoryName(subCategory.getName());

    responseDto.setImageUrl(buildImageUrl(savedProduct.getImagePath()));

    return responseDto;
}

@Override
@Cacheable("productList")
public List<ProductResponseDto> getAllProducts() {

    log.info("Fetching all products");

    List<Product> products = productRepository.findAll();

    log.info("Total products found: {}", products.size());

    return products.stream()
            .map(product -> {

                ProductResponseDto dto = modelMapper.map(product, ProductResponseDto.class);

                dto.setImageUrl(buildImageUrl(product.getImagePath()));

                if (product.getCategory() != null) {
                    dto.setCategoryId(product.getCategory().getId());
                    dto.setCategoryName(product.getCategory().getName());
                }

                if (product.getSubCategory() != null) {
                    dto.setSubCategoryId(product.getSubCategory().getId());
                    dto.setSubCategoryName(product.getSubCategory().getName());
```

```
            }

            return dto;
        })
        .collect(Collectors.toList());
}

@Override
@Cacheable(value = "products", key = "#id")
public ProductResponseDto getProductById(Long id) {

    log.info("Fetching product by id from DB: {}", id);

    Product product = productRepository.findById(id)
        .orElseThrow(() -> {
            log.warn("Product not found with id: {}", id);
            return new ResourceNotFoundException(
                    "Product not found with id: " + id);
        });

    ProductResponseDto dto =
        modelMapper.map(product, ProductResponseDto.class);

    dto.setImageUrl(buildImageUrl(product.getImagePath()));

    if (product.getCategory() != null) {
        dto.setCategoryId(product.getCategory().getId());
        dto.setCategoryName(product.getCategory().getName());
    }

    if (product.getSubCategory() != null) {
        dto.setSubCategoryId(product.getSubCategory().getId());
        dto.setSubCategoryName(product.getSubCategory().getName());
    }

    return dto;
}


@Override
@Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "productList", allEntries = true)
})
public ProductResponseDto updateProduct(Long id,
                                        ProductRequestDto productRequestDto) {
```

```
log.info("Updating product with id: {}", id);

Product existing = productRepository.findById(id)
        .orElseThrow(() -> {
            log.warn("Cannot update. Product not found with id: {}", id);
            return new ResourceNotFoundException(
                    "Product not found with id: " + id);
        });

existing.setName(productRequestDto.getName());
existing.setPrice(productRequestDto.getPrice());
existing.setStock(productRequestDto.getStock());

Category category = categoryRepository
        .findById(productRequestDto.getCategoryId())
        .orElseThrow(() ->
                new ResourceNotFoundException(
                        "Category not found with id: "
                                + productRequestDto.getCategoryId()));

SubCategory subCategory = subCategoryRepository
        .findById(productRequestDto.getSubCategoryId())
        .orElseThrow(() ->
                new ResourceNotFoundException(
                        "SubCategory not found with id: "
                                + productRequestDto.getSubCategoryId()));

if (!subCategory.getCategory().getId()
        .equals(category.getId())) {
    throw new ResourceNotFoundException(
            "SubCategory does not belong to Category");
}

existing.setCategory(category);
existing.setSubCategory(subCategory);

Product updated = productRepository.save(existing);

ProductResponseDto responseDto =
        modelMapper.map(updated, ProductResponseDto.class);

responseDto.setCategoryId(category.getId());
responseDto.setCategoryName(category.getName());

responseDto.setSubCategoryId(subCategory.getId());
responseDto.setSubCategoryName(subCategory.getName());
```

```
    responseDto.setImageUrl(
            buildImageUrl(updated.getImagePath()));

    return responseDto;
}

@Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "productList", allEntries = true)
})
public void deleteProduct(Long id) {

    log.info("Deleting product with id: {}", id);

    Product existing = productRepository.findById(id)
            .orElseThrow(() -> {
                log.warn("Cannot delete. Product not found with id: {}", id);
                return new ResourceNotFoundException("Product not found with id: " + id);
            });

    // ✅ delete primary image from S3
    deleteOldImageIfExists(existing);

    // ✅ delete gallery images from S3
    List<ProductImage> images = productImageRepository.findByProductId(existing.getId());

    for (ProductImage img : images) {
        if (img.getImagePath() != null) {
            s3Service.deleteFileByUrl(img.getImagePath());
        }
        if (img.getThumbnailPath() != null) {
            s3Service.deleteFileByUrl(img.getThumbnailPath());
        }
    }

    productRepository.delete(existing);

    log.info("Product deleted successfully with id: {}", id);
}

@Override
public PagedResponseDto<ProductResponseDto> getAllProducts(
        int page,
        int size,
        String sortBy,
        String sortDir,
        String keyword,
```

```
        Long categoryId
    ) {

        Sort sort = sortDir.equalsIgnoreCase("desc")
                ? Sort.by(sortBy).descending()
                : Sort.by(sortBy).ascending();

        Pageable pageable = PageRequest.of(page, size, sort);

        Page<Product> productPage;

        if (categoryId != null && keyword != null && !keyword.isBlank()) {
            productPage = productRepository.findByCategoryIdAndNameContainingIgnoreCase(categoryId, keyword,
pageable);
        } else if (categoryId != null) {
            productPage = productRepository.findByCategoryId(categoryId, pageable);
        } else if (keyword != null && !keyword.isBlank()) {
            productPage = productRepository.findByNameContainingIgnoreCase(keyword, pageable);
        } else {
            productPage = productRepository.findAll(pageable);
        }

        List<ProductResponseDto> products = productPage.getContent()
                .stream()
                .map(product -> ProductResponseDto.builder()
                        .id(product.getId())
                        .name(product.getName())
                        .price(product.getPrice())
                        .stock(product.getStock())

                        .categoryId(product.getCategory() != null ? product.getCategory().getId() : null)
                        .categoryName(product.getCategory() != null ? product.getCategory().getName() : null)

                        .subCategoryId(product.getSubCategory() != null ? product.getSubCategory().getId() : null)
                        .subCategoryName(product.getSubCategory() != null ? product.getSubCategory().getName() : null)

                        .imagePath(product.getImagePath())
                        .imageUrl(buildImageUrl(product.getImagePath()))
                        .build()
                )
                .toList();

        return PagedResponseDto.<ProductResponseDto>builder()
                .content(products)
                .pageNumber(productPage.getNumber())
                .pageSize(productPage.getSize())
                .totalElements(productPage.getTotalElements())
```

```
            .totalPages(productPage.getTotalPages())
            .last(productPage.isLast())
            .build();
}

@Override
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public ProductResponseDto uploadProductImage(Long productId, MultipartFile file) {

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    validateImageFile(file);

    String imageUrl = s3Service.uploadFile(file, "products/full");

    product.setImagePath(imageUrl);

    Product savedProduct = productRepository.save(product);

    ProductResponseDto dto = modelMapper.map(savedProduct, ProductResponseDto.class);
    dto.setImageUrl(savedProduct.getImagePath());

    return dto;
}

private void validateImageFile(MultipartFile file) {

    if (file == null || file.isEmpty()) {
        throw new InvalidFileException("File is required. Please upload a file.");
    }

    String contentType = file.getContentType();

    if (contentType == null ||
            !(contentType.equals("image/jpeg") ||
                    contentType.equals("image/png"))) {
        throw new InvalidFileException("Only JPG and PNG images are allowed");
    }

    long maxSize = 2 * 1024 * 1024;
    if (file.getSize() > maxSize) {
        throw new InvalidFileException("File size must be less than 2MB");
    }
```

```
}

private void deleteOldImageIfExists(Product product) {

    if (product.getImagePath() == null || product.getImagePath().isBlank()) {
        log.info("No primary image found for product id: {}", product.getId());
        return;
    }

    try {
        s3Service.deleteFileByUrl(product.getImagePath());
        log.info("Old primary image deleted from S3 for product id: {}", product.getId());
    } catch (Exception e) {
        log.warn("Failed to delete old image from S3. Skipping delete. {}", e.getMessage());
    }
}

private String buildImageUrl(String imagePath) {

    if (imagePath == null || imagePath.isBlank()) {
        return baseUrl + "/images/default.jpg";
    }

    // ✅ S3 imagePath already contains full URL
    return imagePath;
}

@Override
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public ProductResponseDto deleteProductImage(Long productId) {

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    if (product.getImagePath() == null || product.getImagePath().isBlank()) {
        throw new InvalidFileException("No image found for this product.");
    }

    // ✅ delete from S3
    s3Service.deleteFileByUrl(product.getImagePath());

    product.setImagePath(null);

    Product savedProduct = productRepository.save(product);
```

```
    ProductResponseDto dto = modelMapper.map(savedProduct, ProductResponseDto.class);
    dto.setImageUrl(null);

    return dto;
}

@Override
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public ProductResponseDto updateProductImage(Long productId, MultipartFile file) {

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    validateImageFile(file);

    // ✅ delete old primary image from S3
    deleteOldImageIfExists(product);

    // ✅ upload new image to S3
    String imageUrl = s3Service.uploadFile(file, "products/full");

    product.setImagePath(imageUrl);

    Product savedProduct = productRepository.save(product);

    ProductResponseDto dto = modelMapper.map(savedProduct, ProductResponseDto.class);
    dto.setImageUrl(savedProduct.getImagePath());

    return dto;
}

@Override
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public List<ProductImageResponseDto> uploadProductImages(Long productId, List<MultipartFile> files) {

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    if (files == null || files.isEmpty()) {
        throw new InvalidFileException("Please upload at least one image.");
```

```
    }

    long existingCount = productImageRepository.countByProductId(productId);

    if (existingCount + files.size() > MAX_GALLERY_IMAGES) {
        throw new InvalidFileException(
                "Maximum " + MAX_GALLERY_IMAGES + " images allowed per product. Already uploaded: "
                        + existingCount + ", trying to upload: " + files.size()
        );
    }

    List<ProductImageResponseDto> responseList = new ArrayList<>();

    for (MultipartFile file : files) {

        validateImageFile(file);

        try {
            ByteArrayOutputStream fullOutputStream = new ByteArrayOutputStream();

            Thumbnails.of(file.getInputStream())
                    .size(800, 800)
                    .outputQuality(0.8)
                    .toOutputStream(fullOutputStream);

            byte[] fullBytes = fullOutputStream.toByteArray();

            ByteArrayOutputStream thumbOutputStream = new ByteArrayOutputStream();

            Thumbnails.of(file.getInputStream())
                    .size(300, 300)
                    .outputQuality(0.7)
                    .toOutputStream(thumbOutputStream);

            byte[] thumbBytes = thumbOutputStream.toByteArray();

            String fullImageUrl = s3Service.uploadBytes(fullBytes, file.getContentType(), "products/gallery/full");
            String thumbImageUrl = s3Service.uploadBytes(thumbBytes, file.getContentType(),
"products/gallery/thumb");

            ProductImage productImage = ProductImage.builder()
                    .imagePath(fullImageUrl)
                    .thumbnailPath(thumbImageUrl)
                    .product(product)
                    .build();

            ProductImage savedImage = productImageRepository.save(productImage);
```

```
            responseList.add(ProductImageResponseDto.builder()
                    .id(savedImage.getId())
                    .imagePath(savedImage.getImagePath())
                    .imageUrl(savedImage.getImagePath())
                    .thumbnailPath(savedImage.getThumbnailPath())
                    .thumbnailUrl(savedImage.getThumbnailPath())
                    .build());

        } catch (Exception e) {
            throw new InvalidFileException("Failed to upload image to S3.");
        }
    }

    if (!responseList.isEmpty()) {
        ProductImageResponseDto latestUploadedImage = responseList.get(responseList.size() - 1);

        product.setImagePath(latestUploadedImage.getImagePath());
        productRepository.save(product);

        log.info("Primary image updated to latest uploaded image for product id: {}", product.getId());
    }

    return responseList;
}

@Override
public List<ProductImageResponseDto> getProductImages(Long productId) {

    productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    List<ProductImage> images = productImageRepository.findByProductId(productId);

    return images.stream()
            .map(img -> ProductImageResponseDto.builder()
                    .id(img.getId())
                    .imagePath(img.getImagePath())
                    .imageUrl(img.getImagePath())
                    .thumbnailPath(img.getThumbnailPath())
                    .thumbnailUrl(img.getThumbnailPath())
                    .build())
            .toList();
}

@Override
@Caching(evict = {
```

```
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
    })
    public void deleteProductImageById(Long imageId) {

        ProductImage productImage = productImageRepository.findById(imageId)
                .orElseThrow(() -> new ResourceNotFoundException("Image not found with id: " + imageId));

        Product product = productImage.getProduct();

        boolean isPrimary = product.getImagePath() != null &&
                product.getImagePath().equals(productImage.getImagePath());

        // ✅ delete full image from S3
        if (productImage.getImagePath() != null) {
            s3Service.deleteFileByUrl(productImage.getImagePath());
        }

        // ✅ delete thumbnail from S3
        if (productImage.getThumbnailPath() != null) {
            s3Service.deleteFileByUrl(productImage.getThumbnailPath());
        }

        productImageRepository.delete(productImage);
        log.info("Gallery image deleted from DB with id: {}", imageId);

        if (isPrimary) {

            List<ProductImage> remainingImages =
                    productImageRepository.findByProductIdOrderByCreatedAtDesc(product.getId());

            if (!remainingImages.isEmpty()) {
                product.setImagePath(remainingImages.get(0).getImagePath());
                log.info("Primary image updated to another gallery image for product id: {}", product.getId());
            } else {
                product.setImagePath(null);
                log.info("No gallery images left. Primary image reset to null for product id: {}", product.getId());
            }

            productRepository.save(product);
        }
    }

    @Override
    @Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
```

```
    })
    public ProductResponseDto setPrimaryProductImage(Long imageId) {

        ProductImage productImage = productImageRepository.findById(imageId)
                .orElseThrow(() -> new ResourceNotFoundException("Image not found with id: " + imageId));

        Product product = productImage.getProduct();

        product.setImagePath(productImage.getImagePath());

        Product savedProduct = productRepository.save(product);

        ProductResponseDto dto = modelMapper.map(savedProduct, ProductResponseDto.class);
        dto.setImageUrl(savedProduct.getImagePath());

        return dto;
    }

    @Override
    public PagedResponseDto<ProductImageResponseDto> getProductImagesWithPagination(Long productId, int
page, int size, String sortDir) {

        productRepository.findById(productId)
                .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

        Sort sort = sortDir.equalsIgnoreCase("desc")
                ? Sort.by("createdAt").descending()
                : Sort.by("createdAt").ascending();

        Pageable pageable = PageRequest.of(page, size, sort);

        Page<ProductImage> imagePage = productImageRepository.findByProductId(productId, pageable);

        List<ProductImageResponseDto> images = imagePage.getContent()
                .stream()
                .map(img -> ProductImageResponseDto.builder()
                        .id(img.getId())
                        .imagePath(img.getImagePath())
                        .imageUrl(img.getImagePath())
                        .thumbnailPath(img.getThumbnailPath())
                        .thumbnailUrl(img.getThumbnailPath())
                        .build())
                .toList();

        return PagedResponseDto.<ProductImageResponseDto>builder()
                .content(images)
                .pageNumber(imagePage.getNumber())
```

```
            .pageSize(imagePage.getSize())
            .totalElements(imagePage.getTotalElements())
            .totalPages(imagePage.getTotalPages())
            .last(imagePage.isLast())
            .build();
}

@Override
@Transactional
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public void reserveStock(Long productId, Integer quantity) {

    if (quantity == null || quantity <= 0) {
        throw new IllegalArgumentException("Quantity must be greater than zero");
    }

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

    if (product.getStock() < quantity) {
        throw new IllegalStateException("Insufficient stock for product id: " + productId);
    }

    product.setStock(product.getStock() - quantity);
    product.setReservedStock(product.getReservedStock() + quantity);

    productRepository.save(product);
}

@Override
@Transactional
@Caching(evict = {
        @CacheEvict(value = "products", key = "#productId"),
        @CacheEvict(value = "productList", allEntries = true)
})
public void confirmStock(Long productId, Integer quantity) {

    if (quantity == null || quantity <= 0) {
        throw new IllegalArgumentException("Quantity must be greater than zero");
    }

    Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));
```

```
        if (product.getReservedStock() < quantity) {
            throw new IllegalStateException("Reserved stock is less than quantity for product id: " + productId);
        }

        product.setReservedStock(product.getReservedStock() - quantity);

        productRepository.save(product);
    }

    @Override
    @Transactional
    @Caching(evict = {
            @CacheEvict(value = "products", key = "#productId"),
            @CacheEvict(value = "productList", allEntries = true)
    })
    public void releaseStock(Long productId, Integer quantity) {

        if (quantity == null || quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be greater than zero");
        }

        Product product = productRepository.findById(productId)
                .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId));

        if (product.getReservedStock() < quantity) {
            throw new IllegalStateException("Reserved stock is less than quantity for product id: " + productId);
        }

        product.setReservedStock(product.getReservedStock() - quantity);
        product.setStock(product.getStock() + quantity);

        productRepository.save(product);
    }
}
```

🔴 First: What We Are Solving

We are using **Redis cache** to make product APIs faster.

But when:

- Product updates
- Product deletes
- Stock changes
- Image changes

Database changes, but cache may still contain old data.

So we added **cache invalidation** to keep data correct.

🟢 1. @Cacheable – Reading from Cache

Example

```
@Cacheable(value = "products", key = "#id")
public ProductResponseDto getProductById(Long id)
```

What this means:

- When this method is called,
- Spring first checks Redis.
- If data exists → return from Redis.
- If not → fetch from DB and store in Redis.

Simple Flow:

```
Client → Check Redis
        ↓
   If present → Return
   If not → DB → Save in Redis → Return
```

For Product List:

```
@Cacheable("productList")
public List<ProductResponseDto> getAllProducts()
```

- Entire product list is stored in Redis.
- Next call does not hit DB.

🟡 2. @CacheEvict – Removing Cache

We use this when data changes.

Example 1: updateProduct

```
@Caching(evict = {
   @CacheEvict(value = "products", key = "#id"),
   @CacheEvict(value = "productList", allEntries = true)
})
```

Let's break it down.

Line 1:

@CacheEvict(value = "products", key = "#id")

Meaning:

- Remove single product from Redis.
- Example: products::1

Why?

Because that product was updated in DB.

Line 2:

@CacheEvict(value = "productList", allEntries = true)

Meaning:

- Remove entire product list cache.

Why?

Because list also contains that product.
If we don't clear it, list may show old price or stock.

Why We Use @Caching

Because we need to evict **multiple caches** in one method.

@Caching allows multiple @CacheEvict together.

🟠 3. deleteProduct

Same logic:

```
@Caching(evict = {
    @CacheEvict(value = "products", key = "#id"),
    @CacheEvict(value = "productList", allEntries = true)
})
```

Why?

Because:

- That product should not exist in cache.
- List should not show deleted product.

🔵 4. Stock Methods

Example:

```
@Caching(evict = {
    @CacheEvict(value = "products", key = "#productId"),
    @CacheEvict(value = "productList", allEntries = true)
})
public void reserveStock(...)
```

Why needed?

Because stock changed.

If stock was 10 and now becomes 7:

Without eviction:

Redis → still says 10 ❌
DB → says 7 ✅

So we remove:

- Single product cache
- Product list cache

Next GET call loads fresh stock.

🟣 5. Image Update Methods

Example:

```
@Caching(evict = {
    @CacheEvict(value = "products", key = "#productId"),
    @CacheEvict(value = "productList", allEntries = true)
})
public ProductResponseDto updateProductImage(...)
```

Why?

Because imagePath changed.

If not removed:

Product detail page → old image ❌

So we clear cache.

🟤 6. Why allEntries = true?

Because productList does not use key like id.

It stores full list.

So we clear entire list cache.

⚪ 7. Default Behavior (Important)

By default:

beforeInvocation = false

Meaning:

Cache is cleared **only after method runs successfully**.

If update fails:

- Cache is NOT removed.
- Old correct data remains.

This prevents accidental data loss.

🧠 Final Concept (Very Important)

We implemented **Cache Aside Pattern**.

Meaning:

1. Read → Check cache first
2. Write → Update DB
3. After write → Remove cache
4. Next read → Fresh DB value cached again

📝 Simple Summary You Can Write in Notes

- @Cacheable → Used for reading. Stores data in Redis.
- @CacheEvict → Used when data changes. Removes outdated cache.
- @Caching → Used when multiple caches need to be cleared.
- key = "#id" → Removes specific product.
- allEntries = true → Removes full list cache.
- Stock and image changes also require cache eviction.
- Default behavior clears cache only after successful DB update.

Now your caching layer ensures:

- No stale price
- No stale stock
- No stale images
- No inconsistency between list and details